

Estudo sobre Listas em Python

Acesso a elementos, diferença
entre lista e tupla, métodos de
remoção e operações comuns

Acesso a elementos de lista

- Em Python, as listas são estruturas que armazenam vários valores em uma única variável. Cada valor tem uma posição (índice), que começa em 0.
- Exemplo:
 - `frutas = ['maçã', 'banana', 'laranja']`
 - `print(frutas[0])` # imprime 'maçã'
 - `print(frutas[2])` # imprime 'laranja'
- Também é possível modificar um item:
 - `frutas[1] = 'uva'`
 - `print(frutas)` # ['maçã', 'uva', 'laranja']
- Se tentar acessar um índice que não existe:
 - `print(frutas[5])` # Erro: IndexError

Diferença entre lista e tupla

- Lista (list):
 - - Mutável (pode ser alterada)
 - - Símbolo usado: colchetes []
 - - Usado quando os dados podem mudar
 - - Exemplo: nomes = ['Ana', 'João']
 - - Pode adicionar/remover itens
- Tupla (tuple):
 - - Imutável (não pode ser alterada)
 - - Símbolo usado: parênteses ()
 - - Usado quando os dados são fixos
 - - Exemplo: coordenadas = (10, 20)
 - - Não pode adicionar/remover itens

Exemplos de operações comuns com listas

- **Criação de uma lista:**
 - `minha_lista = [1, 2, 3, 4, 5]`
 - `print(f'Lista original: {minha_lista}')`
- **Comprimento da lista:**
 - `print(f'Comprimento da lista: {len(minha_lista)}')`
- **Acessando elementos por índice:**
 - `print(f'Primeiro elemento: {minha_lista[0]}')`
 - `print(f'Último elemento: {minha_lista[-1]}')`
- **Slicing (fatiamento) da lista:**
 - `print(f'Primeiros três elementos: {minha_lista[:3]}')`
 - `print(f'Elementos a partir do segundo: {minha_lista[1:]}')`
 - `print(f'Elementos do segundo ao quarto: {minha_lista[1:4]}')`
- **Modificando um elemento:**
 - `minha_lista[0] = 10`
 - `print(f'Lista após modificar o primeiro elemento: {minha_lista}')`
- **Adicionando um elemento no final:**
 - `minha_lista.append(6)`
 - `print(f'Lista após adicionar 6: {minha_lista}')`
- **Adicionando um elemento em uma posição específica:**
 - `minha_lista.insert(2, 100)`
 - `print(f'Lista após inserir 100 na posição 2: {minha_lista}')`

Exemplos de operações comuns com listas

- **Removendo um elemento pelo valor:**
 - `minha_lista.remove(3)`
 - `print(f'Lista após remover 3: {minha_lista}')`
- **Removendo um elemento pelo índice (e retornando o valor removido):**
 - `elemento_removido = minha_lista.pop(1)`
 - `print(f'Lista após remover o elemento na posição 1: {minha_lista}')`
 - `print(f'Elemento removido: {elemento_removido}')`
- **Verificando se um elemento está na lista:**
 - `print(f'O número 4 está na lista? {4 in minha_lista}')`
 - `print(f'O número 20 está na lista? {20 in minha_lista}')`
- **Ordenando a lista (em ordem crescente):**
 - `minha_lista.sort()`
 - `print(f'Lista ordenada: {minha_lista}')`
- **Ordenando a lista (em ordem decrescente):**
 - `minha_lista.sort(reverse=True)`
 - `print(f'Lista ordenada decrescente: {minha_lista}')`
- **Copiando uma lista:**
 - `copia_lista = minha_lista.copy()`
 - `print(f'Cópia da lista: {copia_lista}')`
- **Limpando a lista:**
 - `minha_lista.clear()`
 - `print(f'Lista após ser limpa: {minha_lista}')`

Métodos de remoção de itens

- `remove(valor):`
 - - Remove a primeira ocorrência de um valor específico da lista
 - - Usado quando você sabe o conteúdo que quer remover
 - - Erro se o valor não estiver na lista
 - Exemplo: `nomes.remove('Carlos')`
- `pop(posição):`
 - - Remove e retorna o item de uma posição específica (índice)
 - - Usado quando você sabe a posição do item que quer remover
 - - Erro se o índice não existir
 - - Se não passar nenhum índice, remove o último item
 - Exemplo: `nomes.pop(1)`
- `del lista[posição]:`
 - - Apaga o item de uma posição específica
 - - Usado quando você quer remover sem retornar o valor
 - - Erro se o índice não existir
 - - Também pode apagar a lista inteira ou um intervalo
 - Exemplo: `del nomes[0]`